Lab 14 — Praktijklab: de stack configureren zonder herbouwen

Doel: ConfigMaps aanmaken vanuit een manifest en vanuit bestanden, een Secret, ze inspuiten als variabelen en als gemounte bestanden in de API en in een ongewijzigde officiële nginx, en dan observeren wat een wijziging verandert — of niet verandert — in de lopende Pods.

Vereisten — Lab 13 afgewerkt: k3s, `localhost/api:1.0` in het cluster.

Meegeleverde bestanden (`files/`) - `configmap-api.yaml` — `APP_MESSAGE` en `APP_PROFILE` voor de API. - `secret-db.yaml` — het wachtwoord van de database, als `stringData` . - `deployment-api.yaml` — de API: `envFrom` de ConfigMap, het Secret gemount onder `/run/secrets/db` . - `deployment-web.yaml` — `nginx:alpine` ongewijzigd, geconfigureerd door twee gemounte ConfigMaps, plus de Services `web` en `api` . - `nginx-proxy.conf` , `index.html` — de bestanden die ConfigMaps zullen worden.

Stap 1 — ConfigMaps, op twee manieren

```
mkdir -p ~/labo-docker/14 && cd ~/labo-docker/14 && cp <pad-naar-het-lab>/files/* .
kubectl create namespace lab14 ; kubectl config set-context --current --namespace=lab14
kubectl apply -f configmap-api.yaml
kubectl create configmap web-nginx --from-file=default.conf=nginx-proxy.conf
kubectl create configmap web-html --from-file=index.html
kubectl get configmaps
kubectl get configmap api-config -o yaml | sed -n '/^data:/,/^kind/p'
kubectl describe configmap web-nginx | sed -n '/^Data/,/^$/p' | head -8
```

Observeer vier ConfigMaps (waaronder `kube-root-ca.crt` , aanwezig in elke namespace), `api-config` met zijn twee sleutels, en `web-nginx` waarvan de sleutel `default.conf` het volledige nginx-bestand bevat.

Uitleg. Een ConfigMap is een woordenboek: een waarde kan een woord zijn of een bestand van twintig regels. `--from-file=sleutel=bestand` kiest de naam van de sleutel — hier de naam die het bestand in de container zal dragen.

Stap 2 — Een Secret, en wat zijn "versleuteling" waard is

```
kubectl apply -f secret-db.yaml
kubectl get secret db-secret
kubectl get secret db-secret -o yaml | grep -A1 '^data:'
kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d ; echo
kubectl create secret generic demo --from-literal=token=abc --dry-run=client -o yaml | grep -A1
'^data:'
```

Observeer `db-secret Opaque 1` , de waarde `POSTGRES_PASSWORD: UzNjcjN0LXBn` , dan `S3cr3t-pg` teruggegeven door `base64 -d` ; en een manifest geproduceerd zonder iets aan te maken, met `token: YWJj` .

Uitleg. `stringData` wordt door de API server naar base64 omgezet; wie het Secret kan lezen, kan het decoderen. De bescherming komt van RBAC, versleuteling in rust en het feit dat je die YAML nooit in klare tekst commit.

Stap 3 — Inspuiten: variabelen en bestanden

```
kubectl apply -f deployment-api.yaml -f deployment-web.yaml
kubectl rollout status deployment/api --timeout=60s ; kubectl rollout status deployment/web --
timeout=90s
kubectl exec deploy/api -- env | grep -E 'APP_|DB_'
kubectl exec deploy/api -- ls -l /run/secrets/db/
kubectl exec deploy/api -- cat /run/secrets/db/POSTGRES_PASSWORD ; echo
kubectl exec deploy/api -- wget -qO- http://localhost:8080/ ; echo
```

Observeer `APP_MESSAGE=bonjour depuis une ConfigMap`, `APP_PROFILE=staging` (uit `envFrom`) en `DB_PASSWORD_FILE=/run/secrets/db/POSTGRES_PASSWORD` — maar **niet** het wachtwoord in de omgeving; een bestand `POSTGRES_PASSWORD -> ..data/POSTGRES_PASSWORD` alleen-lezen gemount; `S3cr3t-pg` wanneer je het opent; en de API die antwoordt met het bericht van de ConfigMap.

```
kubectl exec deploy/web -- ls -l /etc/nginx/conf.d/ /usr/share/nginx/html/
kubectl port-forward svc/web 18080:80 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/api/ ; echo ; kill %1
```

Observeer in de Pod `web` — de **officiële** image `nginx:alpine` — een `default.conf -> ..data/default.conf` en een `index.html -> ..data/index.html` afkomstig van de ConfigMaps, en de volledige keten die via de proxy antwoordt.

Uitleg. Variabelen voor de API, bestanden voor nginx en voor het secret. Geen image `web:1.0` en geen `hostPath` meer: de configuratie is een object van het cluster, beschikbaar op elke node.

Stap 4 — Een ConfigMap wijzigen

```
kubectl patch configmap api-config -p '{"data":{"APP_MESSAGE":"message modifie"}}'
sleep 5 ; kubectl exec deploy/api -- env | grep APP_MESSAGE
kubectl rollout restart deployment/api ; kubectl rollout status deployment/api --timeout=60s
kubectl exec deploy/api -- env | grep APP_MESSAGE
```

Observeer na de `patch` nog altijd `bonjour depuis une ConfigMap` in de lopende Pods; na `rollout restart`, `message modifie`.

```
kubectl patch configmap web-html -p '{"data":{"index.html":"<h1>index modifie par la
ConfigMap</h1>\n"}}'
sleep 75 ; kubectl exec deploy/web -- cat /usr/share/nginx/html/index.html
```

Observeer, zonder enige herstart, `<h1>index modifie par la ConfigMap</h1>` (reken op een tot twee minuten naargelang de synchronisatie van de kubelet).

Uitleg. Een variabele wordt bij de aanmaak in het proces gekopieerd; een ConfigMap-volume is een link naar een map die de kubelet vervangt. Voor de variabelen: `rollout restart`; voor de

bestanden: geduld — en een applicatie die het bestand opnieuw leest.

Stap 5 — Een sleutel die niet bestaat

```
kubectl patch deployment api --type=json -
p='[{"op":"add","path":"/spec/template/spec/containers/0/env/-","value":
{"name":"APP_PROFILE","valueFrom":{"configMapKeyRef":{"name":"api-config","key":"profil"}}}}]'
sleep 8 ; kubectl get pods | grep api
kubectl describe pods -l app=api | grep "couldn't find key" | tail -1
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
```

Observeer een nieuwe Pod in `CreateContainerConfigError` naast de twee oude `Running`, en in zijn Events `Error: couldn't find key profil in ConfigMap lab14/api-config` ; dan de terugkeer.

Uitleg. De verwijzingen worden bij het aanmaken van de container opgelost: een afwezige sleutel blokkeert de Pod, en de rollout van lab 12 houdt de oude Pods in dienst.

Opruimen

```
kubectl delete namespace lab14
kubectl config set-context --current --namespace=default
kubectl get configmaps -A | grep -c lab14
```

Observeer `0`. ConfigMaps, Secret, Deployments en Services zijn met de namespace verdwenen. De bestanden van `~/labo-docker/14/` dienen als basis voor lab 15, dat de database toevoegt.

Wat je nu moet kunnen beweren

- `apply -f`, `--from-file`, `--from-literal`, `--dry-run=client -o yaml`: vier manieren om ConfigMaps en Secrets aan te maken.
- `stringData` wordt `UzNjcjN0LXBn` ; `base64 -d` geeft `S3cr3t-pg` : geen versleuteling.
- `envFrom` → `APP_MESSAGE` in `env` ; volume `secret` → `/run/secrets/db/POSTGRES_PASSWORD` , afwezig in de omgeving.
- Officiële `nginx:alpine` geconfigureerd door twee gemounte ConfigMaps: `default.conf -> ..data/default.conf` .
- Gewijzigde ConfigMap: variabele ongewijzigd tot `rollout restart` , bestand vanzelf bijgewerkt na een tot twee minuten.
- Ontbrekende sleutel: `CreateContainerConfigError` , `couldn't find key profil` , oude Pods intact.